

July, 2026



INT_4IP32_TP

Internship report

Final Report

Student's name: TRAN Thanh-Nam

Internship period: 22 April 2026 - 22 July 2026

Multi-task learning for pretraining EEG Foundation models

Abstract

Data on electrical brain dynamics yielded by electroencephalography (EEG) serves as a powerful resource for scientific inquiry and clinical diagnostics. A recent trend in this field involves EEG foundation models, which leverage self-supervised learning to learn robust EEG representations without relying on labels. However, most existing EEG foundation models rely on a single self-supervised task, i.e, masked reconstruction, which limits their ability to capture the diverse features present in EEG data. While multi-task learning offers a promising direction to address this limitation, it often suffers from poor performance in practice due to gradient conflicts between self-supervised tasks. To address this issue, this project applies multi-objective optimization (MOO) methods to mitigate gradient conflicts in multi-task pretraining of EEG foundation models. To validate the effectiveness of this approach, experiments were conducted on downstream tasks, along with an analysis of the pretraining behavior of the model. Experimental results demonstrate that combining multi-task learning with gradient-balancing MOO methods improves performance in both pretraining and downstream tasks compared to loss-balancing methods and their baselines.

Key words: EEG Foundation Models, Self-supervised Learning, Multi-Objective Optimization, Multi-Task Learning.

The source code for this project is available at:

<https://github.com/ttnamnktp/Multi-task-Learning-for-EEGFM.git>

(Note: Please request access or provide your GitHub username to view the repository.)

1 Introduction

Electroencephalography (EEG) is a widely used neurophysiological modality that measures the brain's electrical activity with high temporal resolution, capturing complex neural dynamics across spatial, temporal, and spectral dimensions. It plays a central role in both clinical diagnostics, such

as seizure detection [1], sleep staging [2], and in research applications including brain-computer interfaces (BCI). While traditional supervised deep learning methods for EEG [3, 4] have achieved strong results on individual tasks, they remain fundamentally limited by their reliance on scarce, expert-labeled data and by poor generalizability across subjects, recording sessions, and devices, since models are typically trained from scratch on a single dataset.

To address these limitations, there is a rapidly growing trend towards self-supervised EEG foundation models [5], trained on massive volumes of unlabeled raw EEG data to learn general, robust, and transferable feature representations. Several representative models illustrate this direction. BENDR [6] employs masked autoencoding combined with contrastive learning to pretrain on extensive EEG datasets, while BIOT [7] discretizes each EEG channel into fixed-length segments to enable tokenization across varying channel counts and temporal lengths for transformer-based decoding. LaBraM [8] applies vector-quantized neural spectrum prediction to build a neural tokenizer adaptable to heterogeneous EEG datasets, and EEGPT [9] achieves competitive performance across multiple downstream tasks through self-supervised pretraining that combines contrastive and reconstruction losses. CBraMod [10] incorporates asymmetric conditional positional encoding into a criss-cross transformer architecture. However, a key characteristic of the current landscape is that most EEG foundation models rely on a single, primary self-supervised task, typically masked reconstruction of temporal EEG sequences, to build their feature encoders.

The idea of combining multiple learning objectives through multi-task learning (MTL) has been extensively studied in computer vision [11], natural language processing [12], and reinforcement learning [13]. The most intuitive strategy which is optimizing a simple average of all task losses, often performs poorly in practice, because the resulting average gradient can be small in magnitude or point in a direction that is locally negative for some tasks, leaving them under-optimized. This phenomenon, commonly referred to as gradient conflict, has motivated the development of multi-objective optimization (MOO) methods for deep learning that explicitly account for inter-task gradient interactions.

Broadly, existing approaches fall into two directions [14]. Loss-balancing methods (e.g., OGR [15], Uncertainty Weighting [16]) adjust the relative weight of each task’s loss based on training signals such as loss magnitude or its rate of change; these methods are computationally cheap since they require only a single backward pass. Gradient-balancing methods (e.g., MGDA [17], PCGrad [18], CAGrad [19], NashMTL [20]) instead operate directly on the task gradients, seeking a combined update direction that avoids conflicts between tasks; while these methods tend to yield stronger empirical performance, they are typically more computationally expensive, requiring per-task backward passes and, in some cases, memory that scales with the number of tasks.

The objective of this work is to answer the question:

(Q) How can EEG foundation models be pretrained with multi-task learning while minimizing gradient conflicts efficiently and at low computational cost?

Thus, the contribution of this work including:

- Formulating the gradient conflict problem in EEG foundation model pretraining mathematically, and demonstrating it empirically.
- Applying several MOO methods (OGR [15], FAMO [21]) to mitigate gradient conflicts in EEG foundation model pretraining, and analyzing their behavior theoretically.
- Validating their empirical performance on different downstream tasks by building a PyTorch-based framework for pretraining EEG foundation models with multi tasks.

2 Preliminary

This section provides the background self-supervised learning techniques, then discuss MOO methods for multi-task learning.

2.1 Self-supervised learning for pretraining foundation models

Pretraining framework.

The training module consists of a shared encoder θ and multiple learning tasks, which in this work are the masked reconstruction task and the contrastive task. First, a batch of raw EEG data $x^{raw} = [x_1^{raw}, x_2^{raw}, \dots, x_B^{raw}] \in \mathbb{R}^{B \times C \times t}$ (where B is the batch size, C is the number of channels, and t is the number of data points) is partitioned into patched samples $x = [x_1, x_2, \dots, x_B] \in \mathbb{R}^{B \times N \times C \times T}$, where $N = t/T$ represents the number of patches (or the sequence length) and T is the patch size. The samples x are augmented and masked to obtain $\tilde{x} \odot \mathcal{M}$, where $\mathcal{M} = [\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_B] \in \{0, 1\}^{B \times N \times C}$ denotes the mask matrix for the batch. This masked input is then fed into the encoder, yielding the representation $\tilde{z} = \theta(\tilde{x} \odot \mathcal{M}) \in \mathbb{R}^{B \times N \times M \times D}$, where M is the number of embeddings and D is the representation dimension. Finally, the representation $\tilde{z}$ is utilized for each task to compute the reconstruction loss L_{rec} and the contrastive loss L_{con} , respectively.

Masked reconstruction. Masked reconstruction is widely used in diverse field from computer vision [22], natural language processing [23] and even recent EEG foundation models [10, 9]. The masked reconstruction module reconstruct $x^{rec} = \phi^{rec}(\tilde{z})$. The reconstruction loss for the input batch is computed as

$$L_{rec} = \frac{1}{2B} \sum_{i=1}^B |(x_i^{rec} - x_i) \odot (1 - \mathcal{M}_i)|^2 \quad (1)$$

Contrastive learning Contrastive learning [24] teaches a model to understand data by pulling similar (positive) samples closer together in the representation space while pushing dissimilar (negative) samples farther apart. In the contrastive module, the sample x is augmented and masked into $\hat{x} \odot \hat{\mathcal{M}} \neq \tilde{x} \odot \mathcal{M}$. A target encoder θ_{target} is a copy of θ and learn through EMA (exponential moving average) and its corresponding representation is denoted as $\hat{z} = \theta_{target}(\hat{x} \odot \hat{\mathcal{M}})$. Following the work from [7] using an MLP projector ϕ_{proj} and an MLP predictor ϕ_{pred} in the online branch and a single MLP projector $\hat{\phi}_{proj}$ on the target branch, the output of the online branch and the EMA branch is denoted as

$$\tilde{h}_i = \theta_{pred}(\theta_{proj}(\tilde{z}_i)) \in \mathbb{R}^{B \times D_c} \quad \forall i \in \{1, 2, \dots, B\}, \quad (2)$$

$$\hat{h}_i = \theta_{proj}(\hat{z}_i) \in \mathbb{R}^{B \times D_c} \quad \forall i \in \{1, 2, \dots, B\}, \quad (3)$$

with D_c being the output dimension of the predictor.

The InfoNCE loss function [7] is employed to penalize the model when the similarity of positive pairs falls below that of negative pairs. Here, augmented views originating from the same sample are defined as positive pairs $(\tilde{h}_i, \hat{h}_i)$, whereas views derived from distinct samples constitute negative pairs $(\tilde{h}_i, \hat{h}_j)$ for all $j \neq i$. Initially, by treating the views from the online branch $\{\tilde{h}_i\}_{i=1}^B$ as the anchors, the one-way sample-level InfoNCE loss is formulated as:

$$L_c^{(1)} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(\tilde{h}_i, \hat{h}_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(\tilde{h}_i, \hat{h}_j)/\tau)}, \quad (4)$$

where $\text{sim}(\cdot, \cdot)$ denotes the cosine similarity function, and τ represents the temperature parameter. Symmetrically, the one-way InfoNCE loss taking the views $\{h_i\}_{i=1}^B$ as anchors is defined as:

$$L_c^{(2)} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(\hat{h}_i, \tilde{h}_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(\hat{h}_i, \tilde{h}_j)/\tau)}. \quad (5)$$

To prevent any bias toward a single branch, the final bidirectional contrastive loss is computed as:

$$L_c = \frac{1}{2} (L_c^{(1)} + L_c^{(2)}). \quad (6)$$

2.2 Multi-task learning

When pretraining the encoder θ with 2 different learning tasks namely masked reconstruction and contrastive learning, the objective is to perform these 2 tasks effectively, where each task $i \in \{rec, con\}$ is associated with a loss function $L_i(\theta, x) \in \mathbb{R}_{\geq 0}$. A standard approach involves minimizing the average loss across all tasks:

$$\min_{\theta} \left\{ L_0(\theta, x) = \frac{1}{k} \sum_{i=1}^k L_i(\theta, x) \right\}. \quad (7)$$

Optimization Problem: Direct optimization of (7) often leads to severe under-optimization for a subset of tasks. This challenge is primarily driven by two critical phenomena: gradient conflicts and gradient dominance as illustrated in Figure 1.

At any time step t , assuming the model parameters are updated via a gradient-descent-style iterative scheme: $\theta_{t+1} = \theta_t - \alpha d_t$, where α denotes the learning rate and d_t represents the update vector at time t . Under this setup, a conflict gradient event is said to occur if the update direction opposes the gradient of at least one task:

$$\exists i, \quad \nabla L_i(\theta_t, x_t)^\top d_t < 0. \quad (8)$$

This condition implies that the loss of certain tasks is actively increasing.

Alongside conflicts, gradient dominance presents another major challenge during pretraining models. This occurs when the projections of the update vector d_t onto the gradients of different tasks are highly imbalanced; specifically, certain tasks exhibit exceptionally large projections, while others receive marginal updates due to disproportionately small projections:

$$\exists i, j, \quad |\nabla L_i(\theta_t, x_t)^\top d_t| \gg |\nabla L_j(\theta_t, x_t)^\top d_t|. \quad (9)$$

Consequently, the learning progress of the dominated tasks is severely stifled.

While both gradient conflict and dominance are common during multi-task optimization and are not inherently detrimental in isolation, they become highly undesirable when a subset of tasks suffers from persistent under-optimization as a consequence. In a broader sense, it is disadvantageous when a subset of tasks exhibits significantly slower learning progress compared to the rest, even if all individual task losses are technically decreasing. This combined bottleneck, has motivated previous research aimed at mitigating its effects at each stage of the optimization process.

Pareto dominance To evaluate the efficacy of multi-task learning algorithms in overcoming these optimization bottlenecks during pre-training, the concept of Pareto Dominance serves as a

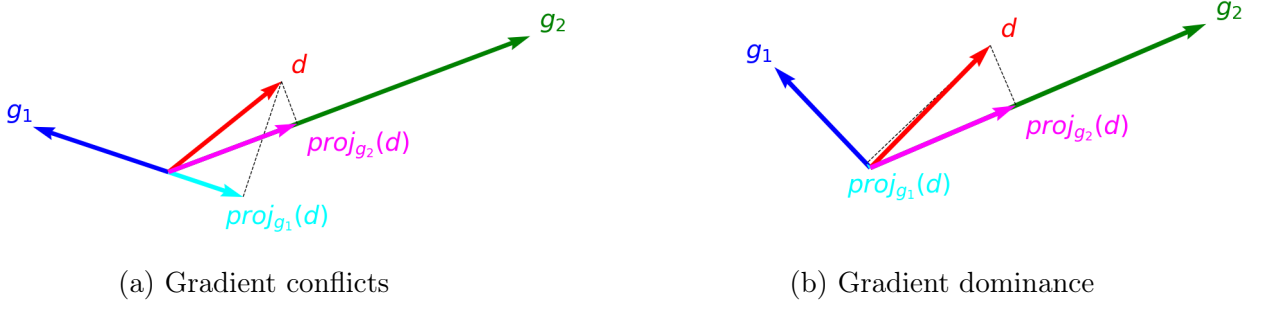


Figure 1: Optimization challenge

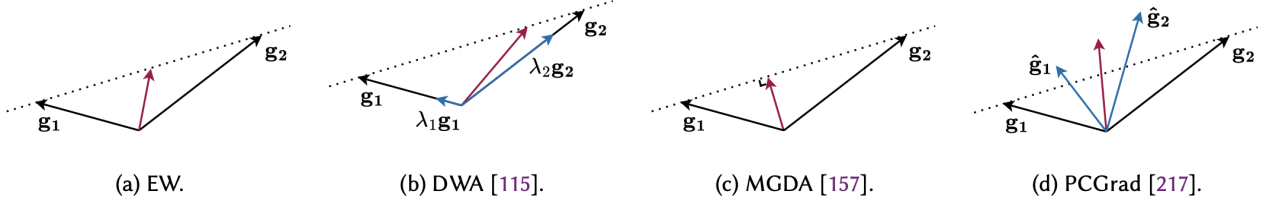


Figure 2: Approaches of MOO methods.

rigorous benchmark. Formally, a parameter state θ is said to Pareto-dominate another state θ' if the loss of all tasks under θ is no worse than under θ' , and at least one task achieves a strictly lower loss:

$$\forall i \in \{1, \dots, k\}, L_i(\theta) \leq L_i(\theta') \quad \text{and} \quad \exists j \in \{1, \dots, k\}, L_j(\theta) < L_j(\theta'). \quad (10)$$

In the context of model pre-training, a superior MOO optimization algorithm is characterized by its ability to guide the model toward a Pareto-dominant loss profile at convergence compared to baseline methods. Specifically, if an algorithm yields a final loss state that Pareto-dominates the state found by another algorithm, it provides a pareto-improving signal. This demonstrates that the algorithm has successfully navigated both gradient conflicts and gradient dominance, allowing all tasks to mutually benefit without sacrificing the performance of any individual task.

Loss balancing methods The Loss Balancing approach addresses multi-objective optimization by dynamically adjusting the loss weights λ_i of individual losses $L_i(\theta, x)$ during training, based on loss characteristics such as rate of descent, or loss scales. It simplifies the optimization back to a linear scalarization form $\min_{\theta} \sum_{i=1}^m \lambda_i L_i(\theta, x)$ where weights λ_i are continuously updated rather than kept static. Typical methods include Dynamic Weight Average (DWA) [25], which estimates weights based on recent loss descent rates, Uncertainty Weighting (UW) [16], which balances tasks by modeling task-dependent homoscedastic uncertainty directly in the objective function, and overfitting-to-generalization ratio alignment (OGR) [15], which dynamically adjusts weights by calculating the degree of overfitting relative to the validation loss. The key advantage of this group is its extremely low computational overhead, which is comparable to equal weighting because it requires only a single backpropagation pass per training step to update parameters. However, the main drawback is the lack of rigorous mathematical guarantees regarding convergence to a Pareto stationary point, and these methods are generally less effective at resolving direct gradient direction conflicts between tasks.

Gradient balancing methods Rather than weighting the loss function, the Gradient Balancing approach operates directly within the gradient space. This method applies optimization algorithms

or geometric projections to determine a unified update direction d by adaptively aggregating the gradient components $g_i = \nabla L_i(\theta)$. Representative methods include the Multiple-Gradient Descent Algorithm (MGDA) [17], which solves a quadratic programming problem to find a common descent direction that maximizes the worst-case improvement across all tasks, and Projecting Conflicting Gradients (PCGrad) [18], which projects conflicting gradients onto the normal plane of other tasks to avoid detrimental interference. The primary advantage of this approach is its superior optimization performance, driven by explicitly addressing directional conflicts at the parameter level, supported by solid theoretical guarantees of convergence to Pareto stationary points. Nonetheless, the approach suffers from high computational and memory costs, as some algorithms requires m independent backward passes to compute individual task gradients before aggregation, making it difficult to scale to pretrain massive neural networks with a large number of data. To mitigate these bottlenecks, Fast Adaptive Multitask Optimization (FAMO) [21] has emerged as a potential solution by leveraging an amortization technique to reduce computational costs, although its efficacy relies on the simplified assumption that the losses change slowly during training.

To answer the question (Q), this project will delve deeper into understanding FAMO’s amortization mechanism and explore the broader potential directions of this approach.

3 Methodology

3.1 Multi-task learning framework for EEG foundation models

This section describes how the multi-task learning (MTL) framework is incorporated into EEG foundation model pretraining. As introduced in Section 2.1, the encoder produces the latent representation $\tilde{z} = \theta(\tilde{x} \odot \mathcal{M}) \in R^{B \times M \times D}$, which is shared by both the masked reconstruction and contrastive learning objectives. After the forward pass, the reconstruction loss L_{rec} and contrastive loss L_{con} are computed. The purpose of the MTL optimization algorithm is to determine a single parameter update direction d that jointly optimizes these objectives.

From a computational perspective, this work categorizes existing MOO methods into two formulations according to how the update direction d is obtained.

The first formulation operates entirely in the loss space by constructing a differentiable surrogate objective

$$L_0 = \Phi(L_{rec}, L_{con}), \quad (11)$$

where $\Phi(\cdot)$ denotes an arbitrary differentiable transformation of the task losses. Depending on the MOO algorithm, Φ may represent a simple weighted summation, adaptive weighting, logarithmic transformation, normalization, or other differentiable scalarization strategies. The update direction is then obtained through a standard backward pass,

$$d = \nabla_{\theta} L_0. \quad (12)$$

An important characteristic of this formulation is that it never explicitly computes the individual task gradients $\nabla_{\theta} L_{rec}$ and $\nabla_{\theta} L_{con}$, requiring only a single backward pass.

The second formulation operates directly in the gradient space. It first computes the gradient of each task independently,

$$g_{rec} = \nabla_{\theta} L_{rec}, \quad (13)$$

$$g_{con} = \nabla_{\theta} L_{con}, \quad (14)$$

and then applies a gradient manipulation operator

$$d = \Psi(g_{rec}, g_{con}), \quad (15)$$

where $\Psi(\cdot)$ represents an arbitrary gradient manipulation strategy designed to produce the final update direction.

The first formulation is computationally efficient because it requires only one backward pass regardless of the number of tasks. Consequently, most loss balancing MOO methods belong to this category, where $\Phi(\cdot)$ often reduces to a weighted combination of the task losses. In contrast, the second formulation is considerably more expensive since each task gradient must be computed separately before any manipulation can be performed. This computational overhead becomes significant during large-scale foundation model pretraining.

Therefore, gradient balancing MOO methods like FAMO that can be reformulated into the first formulation- that is, methods capable of implicitly producing the desired update direction through a surrogate loss without explicitly computing individual task gradients are particularly attractive for foundation model pretraining due to their substantially lower computational cost.

3.2 Overfitting-to-Generalization Ratio (OGR) Alignment

Within the broader landscape of multi-task learning OGR [15] serves as a prominent representative of loss-balancing MOO methods. This approach formulates the multi-task training imbalance as a dynamic multi-objective problem where the goal is to find a set of loss weight that simultaneously minimizes overfitting and maximizes generalization across all input streams.

Let $\mathcal{L}_N^T$ be the training loss and $\mathcal{L}_N^*$ be the target distribution loss (approximated by validation loss $\mathcal{L}_N^V$) at epoch N . The metrics are defined as follows:

- **Overfitting** (O_N): The gap between training improvement and true generalization:

$$O_N \equiv (\mathcal{L}_0^T - \mathcal{L}_N^T) - (\mathcal{L}_0^* - \mathcal{L}_N^*) \quad (16)$$

- **Generalization** (G_N): The actual progress made on the target distribution:

$$G_N \equiv \mathcal{L}_0^* - \mathcal{L}_N^* \quad (17)$$

The blended loss $\mathcal{L}_{blend} = \sum_{i=1}^{k+1} w_i \mathcal{L}_i$ is dynamically weighted using the **GB Estimate** algorithm at epoch N :

$$w_i^* = \frac{1}{Z} \frac{G^i}{(O^i)^2}, \quad \text{where } Z = \sum_{j=1}^{k+1} \frac{G^j}{(O^j)^2} \quad (18)$$

3.3 Fast Adaptive Multitask Optimization (FAMO)

This section presents FAMO, a gradient-balancing MOO method for multi-task learning [21]. There are two key characteristics that make FAMO suitable for pretraining models. First, FAMO chooses the update direction d as a simplex combination of the gradients of the task log-losses. Secondly, FAMO proposes to learn the update weights through an amortization mechanism. These characteristics eliminate the need to calculate the gradient of each task separately as mentioned in 3.1. This project will delve into explaining this mechanism to support this research direction.

3.3.1 Balanced Rate of Loss Improvement

FAMO overcomes the computational bottleneck by reformulating the optimization goal. Instead of balancing the absolute magnitudes of the task gradients, FAMO seeks an update direction d_t that maximizes the worst-case *rate of improvement* across all tasks.

Let the rate of improvement for task i under a model update $\theta_{t+1} = \theta_t - \alpha d_t$ be defined as:

$$r_i(\alpha, d_t) = \frac{L_{i,t} - L_{i,t+1}}{L_{i,t}} \quad (19)$$

FAMO proposes to find d_t by solving the following max-min objective (with an added regularization term):

$$\max_{d_t \in \mathbb{R}^m} \min_{i \in [k]} \frac{1}{\alpha} r_i(\alpha, d_t) - \frac{1}{2} \|d_t\|^2 \quad (20)$$

Applying a first-order Taylor approximation for a small step size $\alpha \rightarrow 0$, FAMO can approximate the rate of improvement by the gradient of the log-loss: $\frac{1}{\alpha} r_i(\alpha, d_t) \approx (\nabla \log L_{i,t})^\top d_t$. This transforms the objective into:

$$\max_{d_t \in \mathbb{R}^m} \min_{i \in [k]} (\nabla \log L_{i,t})^\top d_t - \frac{1}{2} \|d_t\|^2 \quad (21)$$

Using Lagrange duality, the primal high-dimensional optimization problem over $d_t \in \mathbb{R}^m$ can be transformed into a lower-dimensional dual problem over the simplex $\mathbb{S}_k = \{z \in \mathbb{R}^k \mid \mathbf{1}^\top z = 1, z \geq 0\}$ following the proposition 1 in FAMO [21]:

$$z_t^* \in \arg \min_{z \in \mathbb{S}_k} \frac{1}{2} \|J_t z\|^2 \quad (22)$$

where $J_t = [\nabla \log L_{1,t}, \dots, \nabla \log L_{k,t}]^\top \in \mathbb{R}^{k \times m}$ represents the Jacobian matrix of task log-losses, and the optimal update direction is given by $d_t^* = J_t z_t^*$.

Consequently, the optimization problem reduces to finding the weight vector z_t^* at each iteration t that satisfies the dual objective over the simplex $\mathbb{S}_k$. However, solving for z_t^* directly at each step poses a substantial computational burden in practice, particularly when dealing with modern large-scale models where the parameter space m is extremely high-dimensional. To bypass this computational bottleneck, the authors propose an amortization mechanism, which will be detailed in the following section.

3.3.2 Fast Approximation via Amortization

Solving the quadratic program in Eq. (6) at every single training step is computationally expensive and still requires evaluating all task gradients to construct J_t , which leads to k backward passes. To reduce time and space complexity, FAMO introduces an *amortization mechanism*.

Instead of solving the dual problem to convergence at each step, FAMO performs a single-step gradient descent on the combination weights z along the optimization trajectory of the model parameters θ :

$$z_{t+1} = z_t - \beta \tilde{\delta}, \quad \text{where } \tilde{\delta} = \nabla_z \left(\frac{1}{2} \|J_t z\|^2 \right) = J_t^\top J_t z_t \quad (23)$$

Crucially, the gradient $\tilde{\delta} = J_t^\top J_t z_t$ can be approximated without calculating individual task gradients. By using Taylor approximation, FAMO observes the actual change in the task log-losses between two consecutive iterations:

$$\frac{1}{\alpha} \begin{bmatrix} \log L_{1,t} - \log L_{1,t+1} \\ \vdots \\ \log L_{k,t} - \log L_{k,t+1} \end{bmatrix} \approx J_t^\top d_t = J_t^\top J_t z_t \quad (24)$$

To ensure the task weights z_t always remain on the simplex $\mathbb{S}_k$, they are parameterized using unconstrained softmax logits $\xi_t \in \mathbb{R}^k$ such that $z_{i,t} = \text{Softmax}(\xi_{i,t})$. The resulting fast, approximated update on the task logits ξ is given by:

$$\xi_{t+1} = \xi_t - \beta \delta_t \quad (25)$$

where the update signal δ_t is computed using only the temporal change of the log-losses and the Jacobian of the softmax function:

$$\delta_t = \begin{bmatrix} \nabla^\top z_{1,t}(\xi_t) \\ \vdots \\ \nabla^\top z_{k,t}(\xi_t) \end{bmatrix}^\top \begin{bmatrix} \log L_{1,t} - \log L_{1,t+1} \\ \vdots \\ \log L_{k,t} - \log L_{k,t+1} \end{bmatrix} \quad (26)$$

By utilizing this temporal loss history, FAMO completely bypasses the need to compute separate task gradients $\nabla L_{i,t}$. The model parameters θ and the task weights ξ are updated concurrently, maintaining a low computational cost per iteration while dynamically balancing multi-task optimization.

This project tries to investigate the convergence of z_t , thus this work proposed the proposition as follow:

Proposition 1 (Tracking property of FAMO). *When the model learning rate α is sufficiently small, the task weights $z \in \mathbb{S}_k$ generated by FAMO remain close to the instantaneous optimizer*

$$z_t^* \in \arg \min_{z \in \mathbb{S}_k} \frac{1}{2} \|J_t^\top z\|^2,$$

throughout training.

The explanation for the proposition is shown in the Appendix 5.

The update step derived above essentially yields a convex combination composed of log-loss gradients:

$$d^* = \sum_{i=1}^k z_{i,t} \nabla \log L_{i,t} = \sum_{i=1}^k \left(\frac{z_{i,t}}{L_{i,t}} \right) \nabla L_{i,t}. \quad (27)$$

In scenarios where a task loss $L_{i,t}$ becomes extremely small, its corresponding scaling factor $\frac{z_{i,t}}{L_{i,t}}$ can surge dramatically, introducing numerical instability into the optimization process. To mitigate

this issue, we re-scale d^* by a normalizing constant c_t . This adjustment transforms $c_t d^*$ into a proper convex combination of individual task gradients, aligning it with standard gradient manipulation schemes:

$$c_t = \left(\sum_{i=1}^k \frac{z_{i,t}}{L_{i,t}} \right)^{-1}, \quad \text{and the update vector is} \quad d_t = c_t d^* = \sum_{i=1}^k w_i \nabla L_{i,t}, \quad (28)$$

where the normalized task weight is explicitly defined as:

$$w_i = c_t \frac{z_{i,t}}{L_{i,t}}. \quad (29)$$

4 Experimentation

4.1 Experiments setup

Backbone encoders. To assess the effectiveness of the algorithm, extensive tests are conducted using two distinct encoder backbones: CBraMod [10] and EEGPT [9]. CBraMod relies on full-token pretraining over entire latent features. Conversely, EEGPT opts for a more localized approach by pretraining solely on compact summary tokens

Pretraining data The dataset taken from [26] comprises 1,200 hours of high-quality EEG recordings configured under the 10-20 standard system, restricted to 19 major channels: Fp1, Fp2, F7, F3, Fz, F4, F8, T3, C3, Cz, C4, T4, T5, P3, Pz, P4, T6, O1, and O2. To eliminate low-quality signals and noise, the first and last minutes of each recording are discarded, and the remaining signals are processed using a 0.3 Hz–75 Hz band-pass filter alongside a 60 Hz notch filter. These recordings are then resampled to 200 Hz and segmented into non-overlapping 16-second windows. Finally, to mitigate artifacts from non-brain physiological activities, samples with a maximum voltage amplitude exceeding 100 μV are removed, and all remaining segments are normalized to the range of $[-1, 1]$.

Pretraining setup

The detailed hyperparameter configurations for the algorithm components, data augmentation, and pretraining optimization are summarized in Table 1.

Downstream datasets

The generalization capability of pretrained representations is evaluated on five downstream EEG datasets: BCIC-IV-2a [27], PhysioNet-MI [28], KaggleERN [29], and Sleep-EDFx [30]. Since downstream performance is highly dependent on the selection of training, validation, and testing subjects, a cross-subject k -fold evaluation protocol is adopted. For each dataset, subjects are partitioned into k folds, where one fold is reserved for testing (or validation when applicable) and the remaining folds are used for training. The evaluation is repeated k times such that each fold serves as the test set exactly once. Final performance is reported as the average across all folds to ensure a fair and robust comparison.

Downstream evaluation setup

A linear-probe protocol is adopted to rigorously assess the quality of the learned representations. During downstream training, the pretrained encoder is kept frozen, while only the classification head is optimized. Following EEGPT, a trainable 1D convolutional layer is inserted before the

Table 1: Hyperparameter settings for pretraining configuration

Configuration / Component	Value / Specification
Input EEG Specifications	
Channels	19
Time points (16-second samples @ 200Hz)	3200
Patch size	200
Sequence length	16 patches
Data Augmentation	
Temporal Masking	40%
Magnitude scaling	Randomly from 0.5 to 2
Jittering	$\mathcal{N}(0, 0.1)$
Interpolation	20% data points
Frequency masking	20% frequency
Contrastive & Momentum Encoder	
EMA momentum	Cosine ramp (0.996 $\rightarrow$ 1)
Pretraining Optimization	
Total epochs	50
Batch size	128
Optimizer	AdamW
Learning rate	1×10^{-4}
Learning rate schedule	Warmup + Cosine decay
Warmup steps	$0.2T$ (where T is total steps)
Weight decay	0.1 (following a cosine schedule)
Adam β coefficients	(0.9, 0.999)
Adam ϵ	1×10^{-8}
Gradient clipping threshold	1.0
Random seed	7

Table 2: Hyperparameter configuration for downstream tasks

Hyperparameter	Value
Batch size	32
Maximum epochs	50
Algorithm	AdamW
Encoder learning rate	0 (Frozen)
Head learning rate	0.001
Adapter learning rate	0.001
Weight decay	0.01

frozen encoder to adapt pretrained representations to downstream datasets with varying channel configurations. The extracted features are subsequently passed to the classification head. For consistency with the original model architectures, both the EEGPT-style two-layer MLP projection head and the CBraMod classification head are employed for their corresponding pretrained encoders. The downstream parameters are shown in the Table 2.

4.2 Experimental Results

4.2.1 Toy experiments

Although [21] employed toy experiments to simulate the learning process of FAMO, they did not investigate the convergence of z . One contribution of this work is to demonstrate how the amortization process facilitates this convergence through visualization.

In this study, a toy experiment, specifically a 2D multitask optimization problem with two tasks, is designed to visualize and compare how different multi-task learning (MTL) approaches handle conflicting loss objectives. By employing a model with a two-dimensional parameter vector $\theta = (\theta_1, \theta_2) \in \mathbb{R}^2$, the optimization trajectories can be easily plotted and visualized.

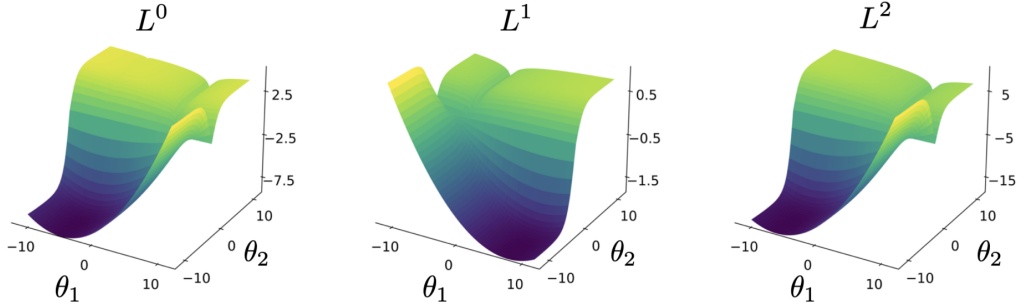


Figure 3: The average loss L_0 and the two task losses L_1 and L_2 for the toy example.

Based on this toy setup, this project provides a clear framework to monitor the dynamics of the task weight vector z throughout the training process. The empirical results demonstrate that the algorithm can find the pareto point, and the amortization process successfully drives z toward a stable configuration as illustrated in Figure 4. This clear convergence of z validates that the network effectively learns to balance the task dynamics rather than oscillating indefinitely, thereby confirming the stability of the amortization mechanism.

4.2.2 Pretraining process

To rigorously evaluate the multi-task learning during the pretraining stage, a comparative analysis is conducted among FAMO, OGR, and Linear Scalarization (LS) which minimizes the average loss. Additionally, single-task learning baselines—specifically Reconstruction-Only and Contrastive-Only—are incorporated to establish the empirical upper bounds for each respective objective. The evaluation mainly focuses on the optimization trajectories, Pareto dominance relationships, and the mitigation of adversarial gradient phenomena, such as gradient conflicts and gradient dominance.

The optimization efficiency is first assessed by monitoring the convergence behavior of both the final reconstruction and contrastive loss values across the training steps. Empirical trajectories reveal that balancing algorithms heavily influence the optimization landscape. Specifically, from

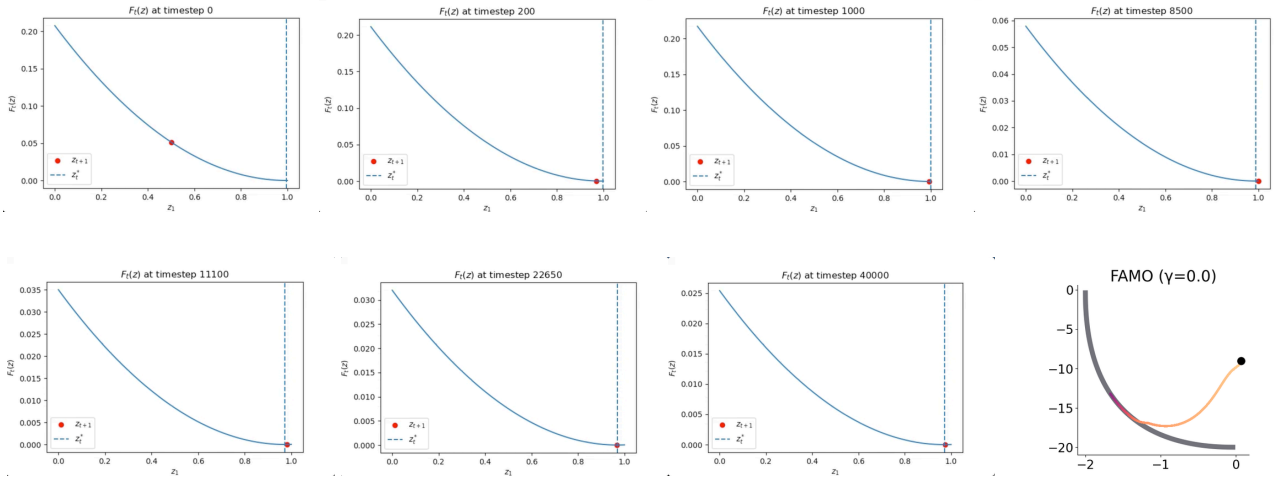


Figure 4: Visualization of the task weight dynamics and convergence behavior. The subplots show $F_t(z)$ across training timesteps, where z_{t+1} progressively aligns with the optimal task weight z_t^* , leading to a stable trajectory on the Pareto front (bottom right).

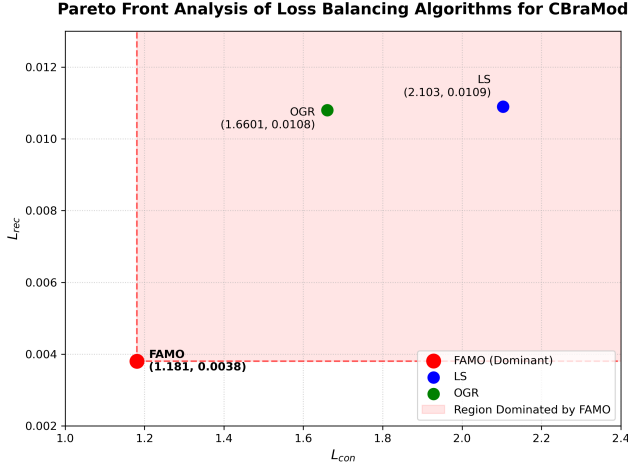
the perspective of Pareto dominance, FAMO demonstrates a superior capacity to simultaneously minimize both objectives without sacrificing one for the other. While LS and OGR frequently suffers from suboptimal trade-offs due to fixed preference weights, the dynamic adjustment mechanism in FAMO successfully guides the optimization path toward a strictly dominating Pareto front solution, as illustrated in Fig. 5.

This performance gap can be further explained by analyzing the gradient dynamics during pretraining (see Fig. 6 and 7). In multi-task frameworks, optimization is inherently challenged by two main phenomena: gradient conflicts, where the gradients of different tasks point in opposing directions ($\cos \theta < 0$) and gradient dominance, where a single task possessing a disproportionately large gradient norm dominates the shared parameter updates.

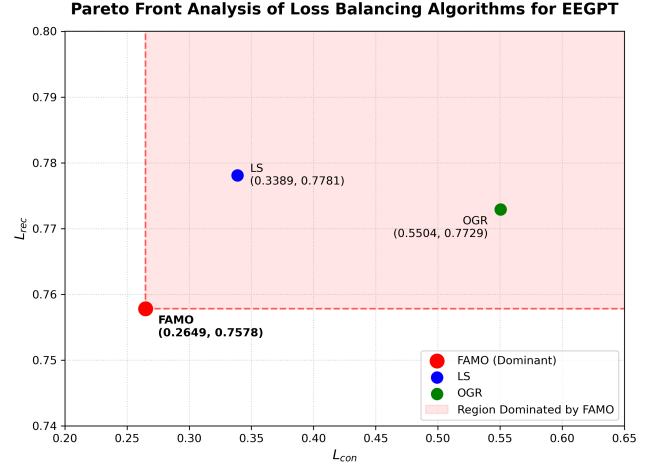
The empirical tracking of gradient cosine similarities and projections shows that both LS and OGR are highly susceptible to these issues, often leading to volatile training or severe degradation in the dominated task. Conversely, FAMO effectively mitigates gradient conflicts and balances task magnitudes, ensuring that the shared representations are learned efficiently and harmoniously from both reconstruction and contrastive objectives.

4.2.3 Results on downstream tasks

To evaluate the quality of the representations learned during the pretraining phase, the models are evaluated on four distinct downstream tasks: BCIC-IV-2a, PhysioNet-MI, KaggleERN, and Sleep-EDFx. Performance comparisons are conducted across two backbone architectures, CBraMod and EEGPT, utilizing five different training strategies: LS, OGR, FAMO, and single-task learning including Reconstruction-only, and Contrastive-only. The empirical results are summarized in



(a) Pareto dominance on CBraMod.



(b) Pareto dominance on EEGPT.

Figure 5: Pareto front analysis of loss balancing algorithms. The red shaded area indicates the region strictly dominated by FAMO.

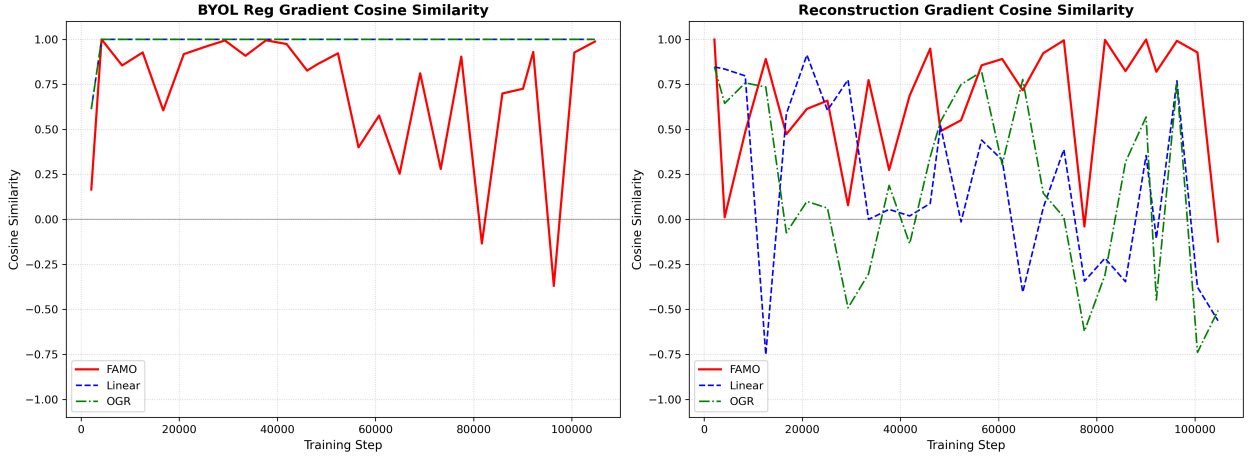


Figure 6: Gradient cosine similarity of the learning tasks relative to the update vector during training for CBraMod, including contrastive learning (left) and masked reconstruction (right).

Table 3.

The experimental outcomes indicate that multi-task optimization strategies significantly influence downstream adaptability. For the CBraMod architecture, FAMO demonstrates strong capabilities, achieving the highest scores on BCIC-IV-2a (56.97%), PhysioNet-MI (57.87%), and Sleep-EDFx (78.46%). Notably, standard multi-task baselines like LS and OGR exhibit clear performance degradation on this backbone, often falling behind the single-task Reconstruction-only baseline. This behavior aligns with the earlier observations of gradient conflicts during pretraining, confirming that poorly balanced gradients heavily compromise the generalizability of the learned features.

Under the EEGPT framework, the performance gap between strategies becomes narrower, yet FAMO retains its competitive edge, particularly on the PhysioNet-MI task, yielding the top performance of 54.42%. While OGR shows a slight advantage on the BCIC-IV-2a and KaggleERN datasets within this backbone, FAMO demonstrates a more robust and well-rounded performance

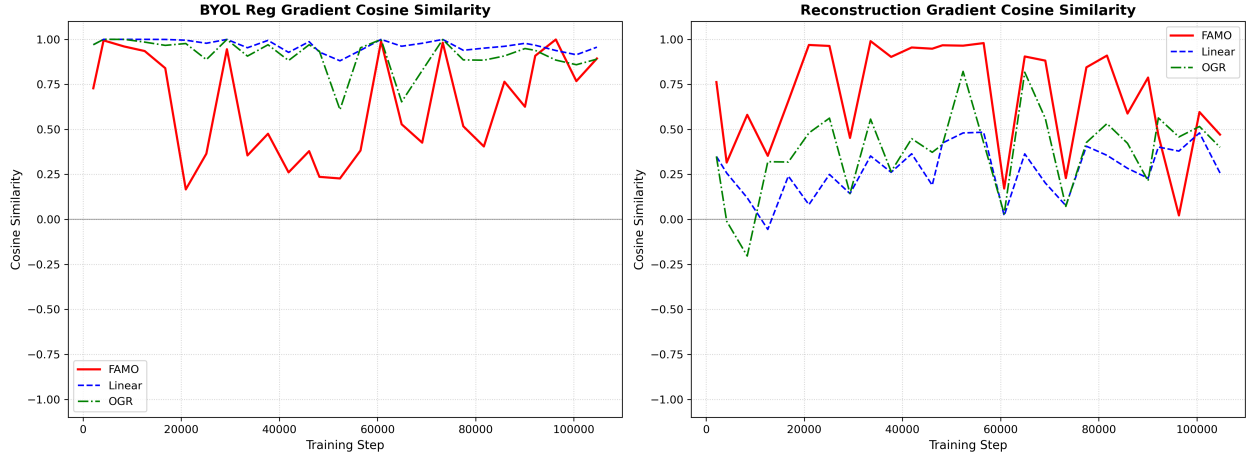


Figure 7: Gradient cosine similarity of the learning tasks relative to the update vector during training for EEGPT, including contrastive learning (left) and masked reconstruction (right).

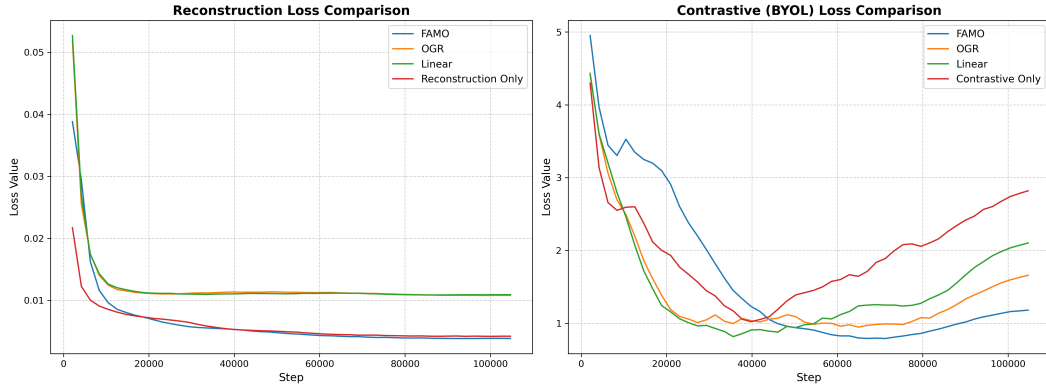


Figure 8: Training loss of CBraMod.

across the majority of the evaluations. Overall, the downstream results validate that dynamically modulating task weights via FAMO effectively yields superior, balanced representations that successfully transfer to diverse target domains.

5 Conclusion

This project has formulated and addressed the problem of pretraining EEG foundation models under a multi-task learning setting, motivated by the observation that existing models rely predominantly on a single self-supervised objective. Within the shared-encoder pretraining framework, two adversarial phenomena were identified as the central optimization challenges: gradient conflict, in which the update direction opposes the gradient of at least one task, and gradient dominance, in which the projections of the update direction onto different task gradients become severely imbalanced. These two phenomena were shown, both theoretically and empirically, to be the primary cause of under-optimization for a subset of tasks during multi-task pretraining, even when the aggregate loss continues to decrease.

To address this problem, existing MOO approaches were categorized into two directions according to how the joint update direction is derived: loss-balancing methods, which construct a differentiable surrogate objective from a weighted combination of task losses and require only a single

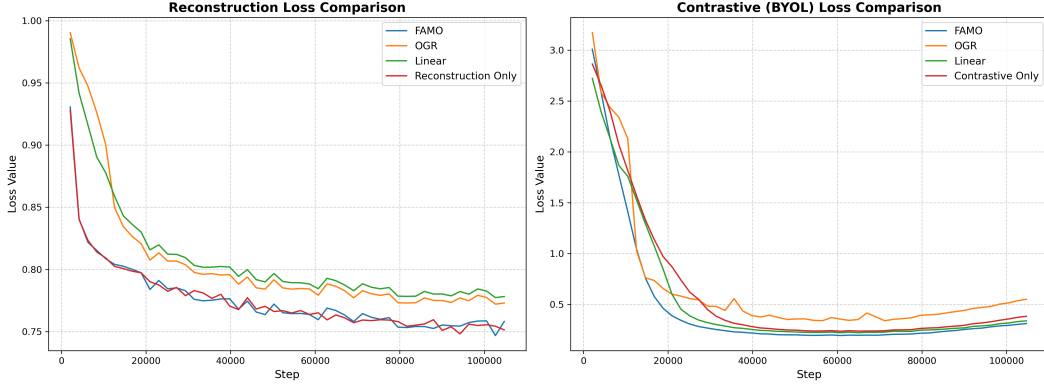


Figure 9: Training loss of EEGPT.

Table 3: Downstream task performance across different pretraining strategies and backbone architectures.

Backbone	Strategy	BCIC-IV-2a	PhysioNet-MI	KaggleERN	Sleep-EDFx
CBraMod	Reconstruction only	<u>54.17</u>	<u>57.48</u>	70.66	73.76
	Contrastive only	45.85	54.30	<u>70.77</u>	<u>73.95</u>
	LS	40.64	51.27	70.79	72.91
	OGR	43.29	51.53	69.54	69.95
	FAMO	56.97	57.87	70.14	78.46
EEGPT	Reconstruction only	51.88	54.09	69.19	77.19
	Contrastive only	52.75	54.07	68.93	78.92
	LS	52.26	<u>54.35</u>	69.52	75.51
	OGR	55.03	54.04	70.32	77.12
	FAMO	<u>53.60</u>	54.42	<u>70.31</u>	<u>78.88</u>

backward pass, and gradient-balancing methods, which directly compute the final update vector. A representative algorithm from each direction was implemented and analyzed, OGR for loss-balancing and FAMO for gradient-balancing with particular attention paid to FAMO’s amortization mechanism and the tracking behavior of its dual weight vector across training. This analysis led to the conclusion that the gradient-balancing direction, by operating directly on task gradients rather than on a scalarized loss, offers a structural advantage in resolving directional conflicts, at the cost of higher computational overhead relative to loss-balancing methods.

Experimental validation on two backbone architectures, CBraMod and EEGPT, pretrained on 1,200 hours of EEG recordings and evaluated on four downstream datasets (BCIC-IV-2a, PhysioNet-MI, KaggleERN, Sleep-EDFx), confirmed this conclusion: FAMO consistently achieved Pareto-dominant loss trajectories during pretraining relative to LS and OGR, and translated this advantage into superior or competitive downstream performance, most notably on the CBraMod backbone. These results confirm that mitigating gradient conflict and gradient dominance directly in the gradient space yields measurably better transferable representations than reweighting task losses alone.

A notable limitation observed during experimentation concerns the computational cost of FAMO’s amortization mechanism. Although FAMO avoids the multiple independent backward passes required by classical gradient-balancing methods such as MGDA and PCGrad, updating its task-weight logits requires evaluating the task losses again after the model parameters have already been updated, in order to observe the resulting change in each log-loss. This means that, in addition

to the forward-backward pass used to compute the parameter update itself, a second forward pass is required at every training iteration solely to refresh the task weights. Empirically, this doubled forward cost increases total pretraining time by approximately 30–50% relative to Linear Scalarization (LS), which requires only a single forward-backward pass per step. Consequently, while FAMO reduces the per-iteration overhead of gradient aggregation to a constant cost with respect to the number of tasks, the hidden cost introduced by this additional forward evaluation remains a bottleneck for scaling to larger EEG foundation models, and constitutes a natural direction for future work aimed at amortizing or approximating this second forward pass.

References

- [1] Athar A Ein Shoka et al. “EEG seizure detection: concepts, techniques, challenges, and future trends”. In: *Multimedia tools and applications* 82.27 (2023), pp. 42021–42051.
- [2] Khald Ali I Aboalayon et al. “Sleep stage classification using EEG signal analysis: a comprehensive survey and new investigation”. In: *Entropy* 18.9 (2016), p. 272.
- [3] Vernon J Lawhern et al. “EEGNet: a compact convolutional neural network for EEG-based brain–computer interfaces”. In: *Journal of neural engineering* 15.5 (2018), p. 056013.
- [4] Yonghao Song et al. “EEG conformer: Convolutional transformer for EEG decoding and visualization”. In: *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 31 (2022), pp. 710–719.
- [5] Gayal Kuruppu et al. “EEG foundation models: a critical review of current progress and future directions”. In: *Journal of neural engineering* 23.2 (2026), p. 021001.
- [6] Demetres Kostas, Stephane Aroca-Ouellette, and Frank Rudzicz. “BENDR: Using transformers and a contrastive self-supervised learning task to learn from massive amounts of EEG data”. In: *Frontiers in Human Neuroscience* 15 (2021), p. 653659.
- [7] Chaoqi Yang, M Westover, and Jimeng Sun. “Biot: Biosignal transformer for cross-data learning in the wild”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 78240–78260.
- [8] Wei-Bang Jiang, Bao-Liang Lu, and Li-Ming Zhao. “LaBraM: A General Large Model for EEG Representation Learning”. In: *AI for Time Series*. CRC Press, pp. 241–270.
- [9] Guagnyu Wang et al. “Eegpt: Pretrained transformer for universal and reliable representation of eeg signals”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 39249–39280.
- [10] Jiquan Wang et al. “Cbramod: A criss-cross brain foundation model for eeg decoding”. In: *International conference on learning representations*. Vol. 2025. 2025, pp. 75310–75346.
- [11] Simon Vandenhende et al. “Multi-task learning for dense prediction tasks: A survey”. In: *IEEE transactions on pattern analysis and machine intelligence* 44.7 (2021), pp. 3614–3633.
- [12] Yufei Wang et al. “Aligning large language models with human: A survey”. In: *arXiv preprint arXiv:2307.12966* (2023).
- [13] Nelson Vithayathil Varghese and Qusay H Mahmoud. “A survey of multi-task deep reinforcement learning”. In: *Electronics* 9.9 (2020), p. 1363.
- [14] Weiyu Chen et al. “Gradient-based multi-objective deep learning: Algorithms, theories, applications, and beyond”. In: *arXiv preprint arXiv:2501.10945* (2025).
- [15] Weiyao Wang, Du Tran, and Matt Feiszli. “What makes training multi-modal classification networks hard?” In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 12695–12705.
- [16] Alex Kendall, Yarin Gal, and Roberto Cipolla. “Multi-task learning using uncertainty to weigh losses for scene geometry and semantics”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 7482–7491.
- [17] Jean-Antoine Désidéri. “Multiple-gradient descent algorithm (MGDA)”. In: (2009).
- [18] Tianhe Yu et al. “Gradient surgery for multi-task learning”. In: *Advances in neural information processing systems* 33 (2020), pp. 5824–5836.

- [19] Bo Liu et al. “Conflict-averse gradient descent for multi-task learning”. In: *Advances in neural information processing systems* 34 (2021), pp. 18878–18890.
- [20] Aviv Navon et al. “Multi-task learning as a bargaining game”. In: *arXiv preprint arXiv:2202.01017* (2022).
- [21] Bo Liu et al. “Famo: Fast adaptive multitask optimization”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 57226–57243.
- [22] Vlad Hondru et al. “Masked image modeling: A survey”. In: *International Journal of Computer Vision* 133.10 (2025), pp. 7154–7200.
- [23] Shitao Xiao et al. “RetroMAE: Pre-training retrieval-oriented language models via masked auto-encoder”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2022, pp. 538–548.
- [24] Haigen Hu et al. “A comprehensive survey on contrastive learning”. In: *Neurocomputing* 610 (2024), p. 128645.
- [25] Shikun Liu, Edward Johns, and Andrew J Davison. “End-to-end multi-task learning with attention”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 1871–1880.
- [26] Iyad Obeid and Joseph Picone. “The temple university hospital EEG data corpus”. In: *Frontiers in neuroscience* 10 (2016), p. 196.
- [27] Clemens Brunner et al. “BCI Competition 2008–Graz data set A”. In: *Institute for knowledge discovery (laboratory of brain-computer interfaces), Graz University of Technology* 16.1-6 (2008), p. 1.
- [28] Ary L Goldberger et al. “PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals”. In: *circulation* 101.23 (2000), e215–e220.
- [29] Perrin Margaux et al. “Objective and Subjective Evaluation of Online Error Correction during P300-Based Spelling”. In: *Advances in Human-Computer Interaction* 2012.1 (2012), p. 578295.
- [30] Bob Kemp et al. “Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG”. In: *IEEE Transactions on Biomedical Engineering* 47.9 (2000), pp. 1185–1194.

Appendix

Tracking analysis of the amortized optimization

The key idea of amortization is to avoid solving the weight optimization problem from scratch at every iteration. Instead, FAMO performs only one gradient update on the task weights. This naturally raises the following question:

Can a single gradient update accurately track the optimal task weights when the optimization objective evolves over time?

To answer the question, this work analyzes the upper bound of the distance between the updated weights and the new optimal weights, denoted as the tracking error $e_{t+1} = \|z_{t+1} - z_{t+1}^*\|$?

Let

$$z_t = \sigma(\xi_t), \quad (30)$$

where $\sigma(\cdot)$ denotes the softmax mapping, and define

$$G_t(\xi) = F_t(\sigma(\xi)), \quad (31)$$

with

$$F_t(z) = \frac{1}{2} \|J_t^T z\|^2. \quad (32)$$

where

$$J_t = \begin{bmatrix} \nabla_{\theta} \log l_1(\theta_t)^T \\ \vdots \\ \nabla_{\theta} \log l_k(\theta_t)^T \end{bmatrix}. \quad (33)$$

After the network parameters are updated according to

$$\theta_{t+1} = \theta_t + \alpha d_t, \quad (34)$$

the optimization objective changes from F_t to F_{t+1} , yielding a new optimal solution

$$\xi_{t+1}^* = \arg \min_{\xi} G_{t+1}(\xi), \quad (35)$$

$$z_{t+1}^* = \arg \min_z F_{t+1}(z) = \sigma(\xi_{t+1}^*). \quad (36)$$

Following the Equation 26, the amortized optimization step is

$$\xi_{t+1} = \xi_t - \eta \nabla G_t(\xi_t) \approx \xi_t - \eta \delta_t. \quad (37)$$

The quality of this amortized update can therefore be measured by the tracking error

$$e_{t+1} = \|z_{t+1} - z_{t+1}^*\|. \quad (38)$$

By adding and subtracting the previous optimum z_t^* , we obtain

$$e_{t+1} = \|z_{t+1} - z_t^* + z_t^* - z_{t+1}^*\|. \quad (39)$$

Applying the triangle inequality yields

$$e_{t+1} \leq \underbrace{\|z_{t+1} - z_t^*\|}_{\text{Optimization error}} + \underbrace{\|z_t^* - z_{t+1}^*\|}_{\text{Optimizer drift}}. \quad (40)$$

The above decomposition reveals two distinct sources of error.

The first term measures the optimization error introduced by performing only one gradient descent step instead of solving the optimization problem exactly. The second term measures the movement of the optimal solution caused by the change of the objective function after updating the network parameters.

We next bound these two terms separately.

Here are some assumptions

Assumption 1 (Interior Simplex Optimum). *The optimum lies in the interior of the simplex*

$$z^* \in \left\{ z \in \mathbb{R}^k \mid z_i > 0, \forall i = 1, \dots, n \text{ v\`a } \sum_{i=1}^n z_i = 1 \right\} \quad (41)$$

Assumption 2 (Local Strong Convexity and Smoothness of G_t). *There exist constants*

$$0 < m_t \leq L_t < \infty$$

such that for every ξ on the line segment joining ξ_t and ξ_t^ ,*

$$m_t I \preceq \nabla^2 G_t(\xi) \preceq L_t I.$$

The local curvature assumption characterizes the geometry of the optimization landscape around the current iterate and the optimum. The lower bound $m_t I \preceq \nabla^2 G_t(\xi)$ ensures that G_t is locally strongly convex, meaning that the objective has sufficient curvature to provide a restoring force towards ξ_t^ . The upper bound $\nabla^2 G_t(\xi) \preceq L_t I$ prevents excessive curvature and guarantees that the gradient varies smoothly in this neighborhood. Together, these conditions ensure that one gradient descent step produces a contraction towards the instantaneous optimum, with the contraction factor controlled by the condition number L_t/m_t .*

Assumption 3 (Smooth Jacobian). *The task-gradient Jacobian is locally Lipschitz with respect to the model parameters θ , namely*

$$\|J(\theta_1) - J(\theta_2)\| \leq L_J \|\theta_1 - \theta_2\|,$$

for some constant $L_J > 0$.

Assumption 4 (Local Strong Convexity of F_t). F_t is locally μ -strongly convex around its minimizer.

The local strong convexity of F_t is equivalent to requiring the Hessian $J_t^\top J_t$ to be locally positive definite. This assumes that the task gradients are not in the same direction.

Step 1. Bounding the optimization error. The first term measures the error introduced by performing only one gradient descent step instead of solving the weight optimization problem exactly. Since both ξ_{t+1} and ξ_t^* are defined with respect to the same objective G_t , this reduces to the contraction property of one gradient descent iteration.

Lemma 1 (Single-step contraction). Under Assumption 1 and 2, if

$$0 < \eta < \frac{2}{L_t},$$

then

$$\|\xi_{t+1} - \xi_t^*\| \leq \rho_t \|\xi_t - \xi_t^*\|,$$

where

$$\rho_t = \max\{|1 - \eta m_t|, |1 - \eta L_t|\} < 1.$$

Corollary 1. Under the assumption 1, the softmax mapping is locally bi-Lipschitz, i.e.,

$$m_\sigma \|\xi - \xi'\| \leq \|\sigma(\xi) - \sigma(\xi')\| \leq L_\sigma \|\xi - \xi'\|.$$

Consequently,

$$\boxed{\|z_{t+1} - z_t^*\| \leq \bar{\rho}_t \|z_t - z_t^*\|},$$

where

$$\bar{\rho}_t = \frac{L_\sigma}{m_\sigma} \rho_t.$$

Therefore, one gradient descent step contracts the error in the logit space. If the learning rate for the weight is small enough ($\bar{\rho}_t < 1$), the updated weight vector w_{t+1} is strictly closer to the instantaneous optimum w_t^* .

Step 2. Bounding the optimizer drift. The second source of error comes from the evolution of the objective after updating the model parameters. Since the task Jacobian depends on θ , updating the network changes the matrix J_t , thereby shifting the optimum of F_t . We next quantify this drift.

Let

$$H_t = J_t J_t^T,$$

so that

$$\nabla F_t(z) = H_t z.$$

Using

$$H_{t+1} - H_t = (J_{t+1} - J_t)J_{t+1}^T + J_t(J_{t+1} - J_t)^T,$$

we obtain

$$\|H_{t+1} - H_t\| \leq (\|J_{t+1}\| + \|J_t\|)\|J_{t+1} - J_t\|.$$

By Assumption 3,

$$\|J_{t+1} - J_t\| \leq L_J \|\theta_{t+1} - \theta_t\|.$$

Since

$$\theta_{t+1} = \theta_t + \alpha d_t,$$

it follows that

$$\|H_{t+1} - H_t\| \leq (\|J_{t+1}\| + \|J_t\|)L_J\alpha\|d_t\|.$$

The remaining factor $(\|J_{t+1}\| + \|J_t\|)\|d_t\|$ is uniformly bounded under FAMO's re-normalization mechanism. Indeed, if the rows of J_t are the log-loss gradients, then

$$\|J_t\| = \mathcal{O}\left(\frac{1}{\sqrt{\ell_{\min,t}}}\right),$$

while FAMO rescales the update direction such that

$$\|d_t\| = \mathcal{O}\left(\sqrt{\ell_{\min,t}}\right).$$

Specifically, recall that the unscaled update direction is $d_t^* = J_t z_t = \sum_{i=1}^k z_{i,t} \nabla \log \ell_{i,t}$, where J_t is the Jacobian matrix of log-losses. Under standard L -smoothness assumptions where task gradients satisfy $\|\nabla \ell_{i,t}\| = \mathcal{O}(\sqrt{\ell_{i,t}})$, the norm $\|d_t^*\|$ scales as $\mathcal{O}(1/\sqrt{\ell_{\min,t}})$. When applying the normalization factor $c_t = \left(\sum_{i=1}^k \frac{z_{i,t}}{\ell_{i,t}}\right)^{-1} = \mathcal{O}(\ell_{\min,t})$, the rescaled update $d_t = c_t d_t^*$ directly yields $\|d_t\| = \mathcal{O}(\ell_{\min,t} \cdot \frac{1}{\sqrt{\ell_{\min,t}}}) = \mathcal{O}(\sqrt{\ell_{\min,t}})$.*

Therefore there exists a constant $M > 0$ satisfying

$$(\|J_{t+1}\| + \|J_t\|)\|d_t\| \leq M,$$

which yields

$$\boxed{\|H_{t+1} - H_t\| \leq ML_J \alpha.}$$

Since z_{t+1}^* minimizes F_{t+1} ,

$$\nabla F_{t+1}(z_{t+1}^*) = 0.$$

Under assumption 4 implies

$$\mu \|z_{t+1}^* - z_t^*\| \leq \|\nabla F_{t+1}(z_t^*)\|.$$

Moreover,

$$\nabla F_t(z_t^*) = 0,$$

thus

$$\begin{aligned} \|\nabla F_{t+1}(z_t^*)\| &= \|(H_{t+1} - H_t)z_t^*\| \\ &\leq \|H_{t+1} - H_t\| \|z_t^*\|. \end{aligned}$$

Hence

$$\|z_{t+1}^* - z_t^*\| \leq \frac{ML_J}{\mu} \alpha \|z_t^*\| \leq \frac{ML_J}{\mu} \alpha.$$

This proves the desired optimizer-drift bound.

Final tracking bound. Finally,

$$\begin{aligned} \|z_{t+1} - z_{t+1}^*\| &\leq \|z_{t+1} - z_t^*\| + \|z_t^* - z_{t+1}^*\| \\ &\leq \bar{\rho}_t \|z_t - z_t^*\| + M \frac{L_J}{\mu} \alpha. \end{aligned} \tag{42}$$

This result shows that the tracking error is governed by two factors: the local contraction of a single amortized optimization step and the temporal variation of the optimization objective. When the learning rate α is small enough that the model parameters evolve sufficiently slowly, the amortized optimization remains close to the instantaneous optimum.